

IFAC PapersOnLine 53-2 (2020) 7033–7040

[illegible]

Automação de código

AutoGenU para Jupyter tem a interface gráfica de usuário não linear (NMPC) com
utilizando JupyterLab e Jupyter Notebook. Utilizamos o pacote de computação
simbólica SymPy para derivar as equações de movimento. Para a otimização, usamos
Broyden-Broyden-Broyden e Jupyter Notebook para a interface gráfica de usuário. JupyterLab
GMRES e o método de Newton para NMPC usando o método de solução em C++.

método de continuação/GMRES (o "GMRES") e o método "GMRES" baseado em disparos múltiplos em C++. O AutoGenU para Jupyter fornece o ambiente de simulação de NMPC com esses solucionadores em C++. O AutoGenU para Jupyter também fornece um ambiente de teste de NMPC rudimentar, com soluções e visualização dos resultados da simulação. O teste também contém exemplos de como se pode utilizar as funções de avaliação de custo e de restrições para a avaliação dos resultados da simulação. O teste também contém exemplos de como se pode utilizar as funções de avaliação de custo e de restrições para a avaliação da elevação de uma vara de carimino usando o AutoGenU para Jupyter. (<http://creativecommons.org/licenses/by-nc-nd/4.0>)

[illegible][illegible][illegible][illegible][illegible][illegible]

Número 15H02257.
 Número 15H02257.
 2405-8963 © 2020, IFAC (International Federation of Automatic Control). Hospedagem por Elsevier Ltd. Todos os direitos reservados.
 Revisão por pares sob responsabilidade da Federação Internacional de Controle Automático. 10.1016/
 i.ifacol.2020.12.447

et al. (2019)) é um software de código aberto para otimização não linear que suporta geração de código C e possui API em Python. Ele também possui uma interface em Python especializada para NMPC, MPCTools (Risbeck e Rawlings (2015)). No entanto, para um uso mais intuitivo, as interfaces Jupyter são preferíveis.

Neste artigo, apresentamos o AutoGenU para Jupyter, uma ferramenta de geração automática de código para NMPC com uma interface amigável e interativa que utiliza JupyterLab e Jupyter Notebook. Usamos o SymPy para a computação simbólica e a geração automática de código C++, incluindo simplificação e eliminação de expressões comuns. Também desenvolvemos solucionadores numéricos baseados em C/GMRES para NMPC em C++ para uso flexível (por exemplo, em robótica). Implementamos não apenas o método C/GMRES original de disparo único, mas também o método C/GMRES baseado em disparos múltiplos com condensação (Shimizu et al. (2009)), que não é fornecido na ferramenta de geração de código existente (Ohtsuka (2015)). Também adicionamos a função Fischer-Burmeister (FB) semi-suave para restrições de desigualdade (Liao-McPherson et al. (2019)) na geração de código. Esta ferramenta fornece o ambiente de simulação para NMPC usando CMake e visualiza os resultados da simulação usando Matplotlib (Hunter (2007)) e seaborn (Waskom M., et al. (2019)).

Este artigo está organizado da seguinte forma. Na Seção 2, discutimos as configurações do problema de NMPC. Na Seção 3, apresentamos os métodos C/GMRES e C/GM-RES baseado em disparos múltiplos. Na Seção 4, apresentamos o AutoGenU para Jupyter e, na Seção 5, fornecemos um exemplo de geração de código e simulação numérica usando o AutoGenU para Jupyter. Na Seção 6, concluímos o artigo.

2. CONFIGURAÇÕES DE PROBLEMAS DE CONTROLE PREDITIVO DE MODELO NÃO LINEAR

Consideramos o sistema não linear $\dot{x}(t) =$

$$f(x(t), u(t), p(t)), \quad (1) \text{ onde } x(t) \in \mathbb{R}^n \text{ denota o vetor de}$$

estado, $u(t) \in \mathbb{R}^{m_u}$ o vetor de entrada de controle e $p(t) \in \mathbb{R}^{m_p}$ o parâmetro variável no tempo. Também consideramos restrições de igualdade de dimensão m_c da forma

$$C(x(t), u(t), p(t)) = 0, \quad (2) \text{ e restrições de desigualdade}$$

de dimensão m_h da forma $h(x(t), u(t), p(t)) \leq 0$, (3) impostas ao sistema

$$(1). \text{ Suponha que (3) seja tratado pela função FB}$$

semi-suave. Observe que as restrições de desigualdade também podem ser transformadas em restrições de igualdade (2) pela introdução de uma entrada fictícia (Ohtsuka (2004)) ou consideradas na função de custo pela introdução de funções barreira (Nocedal e Wright (2006)). No NMPC, um FHOCp do tempo atual t para o futuro finito $t + T$ é resolvido a cada instante de amostragem. Ou seja, queremos encontrar a entrada de controle ótima $u^*(t \rightarrow t + T)$ que minimize a função de custo.

$$J = \int_t^{t+T} L(x(\tau), u(\tau), p(\tau)) d\tau + \phi(x(t+T), p(t+T)) \quad (4)$$

sob (1)–(3), onde $\phi(\cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^{m_p} \rightarrow \mathbb{R}$ denota o custo terminal e $L(\cdot, \cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^{m_u} \times \mathbb{R}^{m_p} \rightarrow \mathbb{R}$ denota o custo de estágio. O intervalo de tempo $[t, t + T]$ é chamado de horizonte do NMPC. Após obter a entrada de controle ótima $u^*(t \rightarrow t + T)$ resolvendo

O problema de controle ótimo fracionário (FHOCp), cujo valor inicial da entrada de controle ótima é $u(t) = u^*(t; x(t))$, é aplicado ao sistema real. Para obter numericamente a entrada de controle ótima, derivamos as condições de Karush-Kuhn-Tucker (KKT), que são as condições necessárias para o controle ótimo. Introduzimos um novo eixo temporal no horizonte $\tilde{y}$ ($0 \leq \tilde{y} \leq T$) e um novo estado no horizonte $x(\tilde{y}; t)$, que denota a trajetória do estado ao longo do eixo $\tilde{y}$ a partir de $x(t)$ em $\tilde{y} = 0$. O FHOCp é então definido encontrando-se a entrada de controle ótima ao longo do eixo $\tilde{y}$ $u(\tilde{y}; t)$ ($0 \leq \tilde{y} \leq T$) que minimiza a função de custo

$$J = \int_0^T \tilde{y} (x(\tilde{y}; t), p(\tilde{y}; t)) d\tilde{y} + \phi(x(T; t), p(T; t)) \quad (5)$$

sujeito a

$$x(0; t) = x(t), \quad (6)$$

$$\frac{d}{d\tilde{y}} x(\tilde{y}; t) = f(x(\tilde{y}; t), u(\tilde{y}; t), p(\tilde{y}; t)), \quad d\tilde{y} \quad (7)$$

$$C(x(\tilde{y}; t), u(\tilde{y}; t), p(\tilde{y}; t)) = 0, \quad (8)$$

e

$$h(x(\tilde{y}; t), u(\tilde{y}; t), p(\tilde{y}; t)) \leq 0. \quad (9)$$

Note que $p(\tilde{y}; t)$ corresponde a $p(\tilde{y} + t)$. Para derivar as condições KKT, introduzimos $\tilde{y} \in \mathbb{R}^n$, $\tilde{y} \in \mathbb{R}^{m_c}$ e $\tilde{y} \in \mathbb{R}^{m_h}$, que denotam os multiplicadores de Lagrange para a equação de estado (7), as restrições de igualdade (8) e as restrições de desigualdade (9), respectivamente. Em seguida, introduzimos o Hamiltoniano definido por $H(x, u, \tilde{y}, \tilde{y}, \tilde{y}, p) := L(x, u, p) + \tilde{y}^T f(x, u, p) + \tilde{y}^T C(x, u, p) + \tilde{y}^T h(x, u, p)$.

$$p. \quad (10)$$

As condições KKT (Bryson e Ho (1975); Nocedal e Wright (2006)) são então dadas como $\tilde{y}(T; t) = \tilde{y}^T$

$$(x(T; t), p(T; t)), \quad (11)$$

$$\tilde{y}(\tilde{y}; t) =$$

$$\tilde{y}^T H_T(x(\tilde{y}; t), u(\tilde{y}; t), \tilde{y}(\tilde{y}; t), \tilde{y}(\tilde{y}; t), \tilde{y}(\tilde{y}; t), p(\tilde{y}; t)), \quad (12)$$

$$H_T(x(\tilde{y}; t), u(\tilde{y}; t), \tilde{y}(\tilde{y}; t), \tilde{y}(\tilde{y}; t), \tilde{y}(\tilde{y}; t), p(\tilde{y}; t)) = 0, \quad (13)$$

$$\tilde{y}(\tilde{y}; t) \leq 0, \quad \tilde{y}^T T(\tilde{y}; t) h(x(\tilde{y}; t), u(\tilde{y}; t), p(\tilde{y}; t)) = 0. \quad (14)$$

Note que uma função escalar com um subscrito denota a derivada parcial da função escalar em relação ao subscrito. As variáveis desconhecidas do FHOCp são dadas por $x(\tilde{y}; t)$, $u(\tilde{y}; t)$, $\tilde{y}(\tilde{y}; t)$, $\tilde{y}(\tilde{y}; t)$ e $\tilde{y}(\tilde{y}; t)$ para $0 \leq \tilde{y} \leq T$, que devem satisfazer (6)–(14). Observe que (9) e (14) denotam a condição de complementaridade entre $\tilde{y}(\tilde{y}; t)$ e $h(x(\tilde{y}; t), u(\tilde{y}; t), p(\tilde{y}; t))$. A função FB semi-suave (Liao-McPherson et al. (2019)) é definida para uma condição de complementaridade a $\tilde{y} = 0$, $b \geq 0$, $ab = 0$ por

$$\tilde{y}(a, b) := \tilde{y}a^2 + b^2 + \tilde{y}^2 \tilde{y} (a + b), \text{ de modo} \quad (15)$$

que possui a propriedade de que $\tilde{y}(a,$

$$b) = 0 \text{ se } a = 0, b \geq 0, \tilde{y} = \tilde{y}^2 2ab, \quad (16) \text{ onde } \tilde{y} \geq 0 \text{ relaxa a}$$

ortogonalidade $ab = 0$ para evitar que $\tilde{y}(a, b)$ seja indistinguível quando $(a, b) = (0, 0)$. Observe que a condição de complementaridade original a $\tilde{y} = 0$, $b \geq 0$, $ab = 0$ é válida quando $\tilde{y} \geq 0$. Consideramos

$$\tilde{y}(x, u, \tilde{y}, p) := \begin{pmatrix} \tilde{y}(\tilde{y}(1), \tilde{y}h(1)(x, u, p)) \\ \vdots \\ \tilde{y}(\tilde{y}(m_h), \tilde{y}h(m_h)(x, u, p)) \end{pmatrix} \cdot \tilde{y} = 0 \quad (17)$$

em vez de (9) e (14), onde $\tilde{y}(i)$ denota o i -ésimo elemento de $\tilde{y}$, $h(i)$ (x , u , p) denota o i -ésimo elemento de $h(x, u, p)$. Observe que (17) é equivalente a (9) e (14) quando $\tilde{y} \approx 0$ e podemos considerar as restrições de desigualdade originais (3) com $\tilde{y} > 0$ suficientemente pequeno.

Para obter numericamente a solução, discretizamos o FHOC. Dividimos o horizonte em N passos e definimos $\tilde{y} := T/N$. Discretizamos o estado no horizonte em $x(0), \dots, x(N)$ e $u(0), \dots, u(N)$, os multiplicadores de Lagrange em $\lambda(0), \dots, \lambda(N)$, $\tilde{y}(0), \dots, \tilde{y}(N)$ e o parâmetro variável no tempo em $p(0), \dots, p(N)$. Observe que $\lambda(i)$ corresponde a $\lambda(i)$ em (8), (11)–(13) e (17) em

0

$x(0) = x(0)$, (18) $x(i+1) = x(i) + f(x(i), u(i), p(i))\tilde{y}$, $i = 0, \dots, N-1$, (19) (20) (21)

$$C(x(i), u(i), p(i)) = 0, i = 0, \dots, N-1, N(t) = \tilde{y}^T$$

$$p(i) = \tilde{y}^{i+1}(t)$$

$$+ H_{T_x}(x(i), u(i), \tilde{y}^{i+1}(t), \tilde{y}(t), \tilde{y}(t), p(i))\tilde{y}, (22)$$

$$i = 1, \dots, N-1,$$

$$H_{T_u}(x(i), u(i), \tilde{y}^{i+1}(t), \tilde{y}(t), \tilde{y}(t), p(i)) = 0, N-1, (23) i = 0, \dots,$$

e

$$\tilde{y}(x(i), u(i), \tilde{y}^{i+1}(t), p(i)) = 0, i = 0, \dots, N-1. (24) u(N), \tilde{y}(N)$$

As variáveis ótimas $x(0), \dots, x(N)$, $u(0), \dots, u(N)$, $\lambda(0), \dots, \lambda(N)$, $\tilde{y}(0), \dots, \tilde{y}(N)$ e $p(0), \dots, p(N)$ devem satisfazer (18)–(24), que definem o TPBVP.

3. ALGORITMOS NUMÉRICOS DE NMPC BASEADOS NO MÉTODO C/GMRES

3.1 Método C/GMRES

O método C/GMRES considera $u(0), \dots, u(N)$, $\tilde{y}(0), \dots, \tilde{y}(N)$ e $p(0), \dots, p(N)$ como variáveis a serem determinadas. Dados $x(0), \dots, x(N)$, $u(0), \dots, u(N)$, $\lambda(0), \dots, \lambda(N)$, podemos determinar $x(0), \dots, x(N)$ a partir de (21) e (22). $u(N)$

 $x(N)$

a partir de (18) e (19) e $\tilde{y}$. Os erros na otimalidade de dados $u(0), \dots, u(N)$, $\tilde{y}(0), \dots, \tilde{y}(N)$ e $p(0), \dots, p(N)$ são então calculados por (23), (20) e (24). Seja $U(t)$ um vetor composto pelas variáveis a serem determinadas e $F(U(t), x(t), t)$ uma função vetorial composta por (23), (20) e (24), ou seja, definimos $U(t)$ e $F(U(t), x(t), t)$ como (25). Em seguida, o FHOC é reduzido a um problema não linear para encontrar $U(t)$ que satisfaça $F(U(t), x(t), t) = 0$. Este método também é chamado de método de disparo único, em contraste com o método de disparo múltiplo, que será apresentado na próxima subseção. Com o método C/GMRES, não resolvemos o problema não linear $F(U(t), x(t), t) = 0$ diretamente, pois isso requer uma busca iterativa ineficiente da solução, como no método de Newton. Em vez disso, o método C/GMRES rastreia $U(t)$ sem tais iterações, calculando a derivada temporal $\dot{U}(t)$ de forma que $U(t)$ satisfaça $F(U(t), x(t), t) = 0$.

Se $U(0)$ satisfaz $F(U(0), x(0), 0) = 0$, então podemos rastrear $U(t)$ que satisfaz $F(U(t), x(t), t) = 0$ integrando $\dot{U}(t)$

satisfazendo as derivadas temporais de $F(U(t), x(t), t) = 0$. Aplicando o método de continuação (Richter e DeCarlo (1983)) a $F(U(t), x(t), t) = 0$, obtemos $\dot{F}(U(t), x(t), t) = \dot{F}(U(t), x(t), t)$,

t , (26) onde $\tilde{y}$ é um valor real positivo. O lado direito

de (26) desempenha um papel na estabilização do cálculo numérico. A equação (26) é ainda transformada em $\dot{U} + F_{xx} + Ft = \dot{F}$, que pode ser vista como um

problema linear de $\dot{U} + F_{xx} + Ft = \dot{F}$ (27)

$$\dot{U} = \dot{F} - F_{xx} - Ft. (28)$$

, por exemplo, como

Note que omitimos os argumentos em (27) e (28). Com o método C/GMRES, os produtos dos jacobianos e vetores em (28) são calculados eficientemente pela aproximação de diferenças finitas, e (28) é resolvido usando o método GMRES (Kelly (1995)), um método computacional rápido para problemas lineares. Após obter $\dot{U}(t)$ resolvendo (28), a solução é atualizada por $U(t + \tilde{y}) = U(t) + \dot{U}(t)\tilde{y}$, (29) onde $\tilde{y} > 0$

é um período de amostragem. O método C/

GMRES resolve (28) apenas uma vez por atualização de $U(t)$, e o custo computacional da atualização de $U(t)$ corresponde a apenas uma iteração com o método de Newton.

Note que podemos ter que obter a solução inicial $U(0)$ que satisfaça $F(U(0), x(0), 0) = 0$ dentro de um determinado período de amostragem. Uma estratégia para calcular $U(0)$ com um tempo computacional curto é definir o comprimento do horizonte como uma função suave dependente do tempo, tal que $T(0) = 0$ e $T(t) \rightarrow T_f(t \rightarrow \tilde{y})$, por exemplo, $T(t) =$

$$T_f(1 - e^{-\tilde{y}t}), (30) U(0) = u(0), \text{ onde } T_f \text{ e } \tilde{y} \text{ são}$$

positivos. Então $u(0) = \tilde{y}(0)$, $\tilde{y}(0) = \tilde{y}(0)$, $x(0) = \dots$ valores $x(0)$ e $\tilde{y}(0) = \tilde{y}$. $\tilde{y}_k(x(0), p(0))$ valem para qualquer i porque $T(0) = 0$. Nós pode obter $u(0)$, $\tilde{y}(0)$ e $\tilde{y}(0)$ resolvendo

$$H_{T_u}(x(0), u(0), \tilde{y}(x(0), p(0)), \tilde{y}(0), p(0))$$

$$C(x(0), u(0), p(0)) = 0 (31)$$

$$\tilde{y}(x(0), u(0), p(0)) = 0$$

com um tempo curto mesmo com o método de Newton porque o problema não linear (31) é suficientemente pequeno.

3.2 Método C/GMRES baseado em disparos múltiplos

Formulação do Problema: Em contraste com o método de disparo único, o método de disparo múltiplo considera todas as variáveis $u(0), \dots, u(N)$, $x(1), \dots, x(N)$, $\tilde{y}(1), \dots, \tilde{y}(N)$ e $p(0), \dots, p(N)$ como variáveis a serem determinadas do problema de valor de contorno de três pontos (TPBVP). Como os erros de otimalidade são distribuídos em todas as variáveis, essa formulação melhora a estabilidade numérica em relação ao método de disparo único, no qual os erros aparecem apenas

$$\tilde{y}(1), \dots, \tilde{y}(N) \text{ e } p(0), \dots, p(N) \text{ em (32) e (33) e o FHOC do}$$

método de disparo múltiplo é reduzido ao seguinte problema não linear: encontrar $U(t)$ definido por (25) e $X(t)$ definido por (32) satisfazendo $F(U(t), X(t), t)$ definido como a mesma forma de (25) e $G(U(t), X(t), t)$ definido como (33).

$$\begin{aligned}
 U(t) := & \begin{pmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_{N_y}(t) \end{pmatrix}, F(U(t), x(t), t) := \begin{pmatrix} HT_u(x_1(t), u_1(t), \ddot{y}_1(t), \ddot{y}_2(t), \ddot{y}_3(t), p_0(t)) \\ C(x_1(t), u_1(t), p_0(t)) \ddot{y}_1(t) \\ 0(t), u_1(t), \ddot{y}_1(t), p_0(t) \\ \vdots \\ HT_u(x_{N_y}(t), u_{N_y}(t), \ddot{y}_{N_y}(t), \ddot{y}_{N_y+1}(t), \ddot{y}_{N_y+2}(t), p_{N_y}(t)) \\ C(x_{N_y}(t), u_{N_y}(t), p_{N_y}(t)) \ddot{y}_{N_y}(t) \\ N_{y1}(t), u_{N_y}(t), \ddot{y}_{N_y}(t), p_{N_y}(t) \end{pmatrix} \quad (25)
 \end{aligned}$$

$$\begin{aligned}
 G(U(t), X(t), x(t), t) := & \begin{pmatrix} x_1(t) \ddot{y}_1(t) - x(t) \ddot{y}_1(t) - f(x(t), u_1(t), p_0(t)) \ddot{y}_1(t) - \ddot{y}_2(t) \\ \ddot{y}_1(t) HT(x_1(t), u_1(t), \ddot{y}_1(t), \ddot{y}_2(t), \ddot{y}_3(t), p_1(t)) \ddot{y}_1(t) \\ \vdots \\ x_{N_y}(t) \ddot{y}_{N_y}(t) - x_{N_y}(t) \ddot{y}_{N_y}(t) - f(x_{N_y}(t), u_{N_y}(t), p_{N_y}(t)) \ddot{y}_{N_y}(t) \\ N_{y1}(t) \ddot{y}_{N_y}(t) - \ddot{y}_{N_y}(t) HT(x_{N_y}(t), u_{N_y}(t), \ddot{y}_{N_y}(t), \ddot{y}_{N_y+1}(t), \ddot{y}_{N_y+2}(t), p_{N_y}(t)) \ddot{y}_{N_y}(t) \\ N(t) \ddot{y}_{N_y}(t) - \ddot{y}_{N_y}(t) HT(x_{N_y}(t), u_{N_y}(t), \ddot{y}_{N_y}(t), \ddot{y}_{N_y+1}(t), \ddot{y}_{N_y+2}(t), p_{N_y}(t)) \ddot{y}_{N_y}(t) \\ \ddot{y}_{N_y}(t) - \ddot{y}_{N_y}(t) HT(x_{N_y}(t), u_{N_y}(t), \ddot{y}_{N_y}(t), \ddot{y}_{N_y+1}(t), \ddot{y}_{N_y+2}(t), p_{N_y}(t)) \ddot{y}_{N_y}(t) \end{pmatrix} = 0. \quad (33)
 \end{aligned}$$

Condensando. Usando o método de continuação, (25) e (33) são transformados em

$$F U \ddot{y} + F X \ddot{y} + F x \ddot{y} + F t = \ddot{y} F, \quad (34)$$

e

$$G U \ddot{y} + G X \ddot{y} + G x \ddot{y} + G t = \ddot{y} G, \quad (35) \text{ e a solução } U \text{ e } X$$

podem ser atualizadas integrando $\ddot{y}$ e X , bem como (29). Por exemplo, podemos obter $\ddot{y}$ e X resolvendo

$$\ddot{y} = \ddot{y} - \ddot{y} F X \ddot{y} - \ddot{y} F x \ddot{y} - \ddot{y} F t + \ddot{y} F t \ddot{y}. \quad (36)$$

No entanto, este problema é mais dispendioso do que o do método C/GMRES original, porque a dimensão do problema linear (36) é maior do que a de (28). Para reduzir o custo computacional na resolução do problema linear da formulação de disparos múltiplos, Shimizu et al. (2009) aplicaram condensação (Bock e Plitt (1984)) e removeram X e parte de $\ddot{y}$ de (36). Note-se que para a condensação de parte de $\ddot{y}$

$$, F \text{ e } U \text{ devem ser particionados como } F = [F^T \quad F^-]^T$$

$$F^T]^T, U = [U^T \quad U^-]^T \quad (37) \text{ de modo que as três condições}$$

seguintes sejam satisfeitas: 1) F^- e U^- têm o mesmo tamanho. 2) $G U^- = 0$ e $F^- X = 0$ são válidas. 3) O produto de $F^- \ddot{y}_1$ e um vetor arbitrário pode ser obtido com baixo custo computacional. Sob 1) — U^- de (36). Por exemplo, considere 3), podemos remover a saturação na entrada de controle, ou seja, $u_{\min} \leq u_j(t) \leq u_{\max}$ elemento restrito de u . Ao $\ddot{y}_{\max}$, (38) onde $j \in \{1, 2, \dots, \mu\}$ especifica o introduzir uma entrada fictícia $\hat{u} \in R$, as restrições

$$\text{de desigualdade (38) são transformadas na restrição de}$$

igualdade.

$$u_j^2 - u_{j\max}^2 - u_{j\min}^2 = 0. \quad (39)$$

A entrada fictícia é tratada como uma variável a ser determinada no TPBVP. Para considerar (39), um multiplicador de Lagrange para (39), $\ddot{y} \in R$, também é adicionado às variáveis a serem determinadas. Então (39) satisfaz as três condições acima para condensação, e podemos remover as derivadas temporais de u^- e $\ddot{y}$ da solução do problema linear.

4. AUTOGENU PARA JUPYTER

4.1 Visão geral do AutoGenU para Jupyter

AutoGenU para Jupyter (Katayama (2018-2020)) é uma ferramenta de software de código aberto para NMPC. É composta principalmente por três partes:

- AutoGenU.ipynb: Interface principal do AutoGenU para Jupyter (o arquivo .ipynb pode ser renomeado como desejarmos).
- autogenu: Diretório contendo módulos Python usados no AutoGenU.ipynb para geração automática de código C++, compilação de arquivos fonte C++, execução e visualização de simulações numéricas.
- include/cgmres, src: Diretório contendo arquivos de cabeçalho C++ e arquivos fonte dos métodos C/GMRES e C/GMRES baseado em disparos múltiplos. Inclui também um subdiretório contendo arquivos de cabeçalho C++ e arquivos fonte para simulações numéricas.

Por meio do AutoGenU.ipynb, podemos gerar arquivos-fonte C++ com configurações de problemas NMPC, compilar esses arquivos, executar simulações numéricas e gerar gráficos com os resultados das simulações. Os módulos Python do AutoGenU são desenvolvidos especificamente para ele, ou seja, sob a premissa de que os usuários o utilizarão por meio do AutoGenU.ipynb. Por outro lado, as bibliotecas C++ podem ser usadas sem o AutoGenU.ipynb, desde que os usuários preparem os arquivos-fonte C++ com as configurações dos problemas NMPC. Portanto, explicamos todos os fluxos de trabalho do AutoGenU.ipynb e das bibliotecas C++. Observe que assumimos que o AutoGenU.ipynb, o AutoGenU, o diretório `include` e o diretório `src` estejam no mesmo diretório de trabalho. Também assumimos que o Python 3.6 ou posterior, o Jupyter Notebook ou JupyterLab, o SymPy, o NumPy, o Matplotlib, o Seaborn, o compilador C++11 e o CMake estejam instalados no ambiente.

Fluxos de trabalho do AutoGenU A Figura 1 ilustra uma visão geral dos fluxos de trabalho do AutoGenU para Jupyter. Esses fluxos de trabalho podem ser divididos em cinco partes. A primeira parte é a “descrição do problema”, na qual os usuários devem inserir as dimensões x , u , $C(x, u, p)$ e $h(x, u, p)$ e definir as funções simbólicas $f(x, u, p)$, $C(x, u, p)$, $h(x, u, p)$, $L(x, u, p)$ e $\ddot{y}(x, p)$. A segunda parte, “computação simbólica”, deriva o TPBVP com base nas funções simbólicas definidas na parte anterior. A terceira parte é a “geração de código”, que gera o código-fonte em C++.

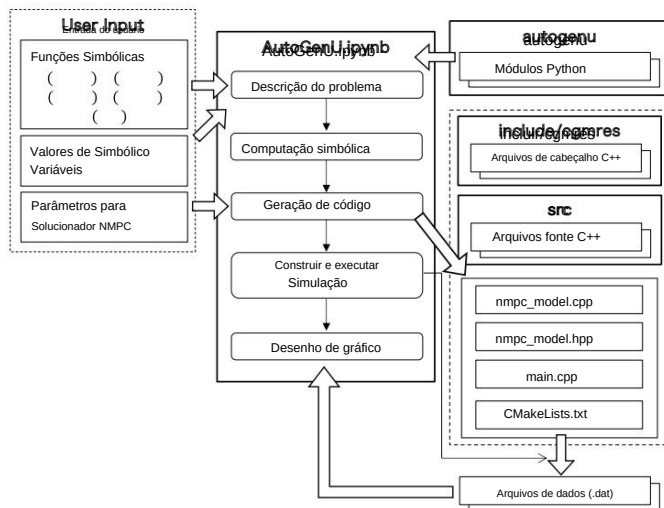


Figura 1. Visão geral do AutoGenU para Jupyter

Os arquivos contêm as configurações do problema NMPC. Nesta parte, os usuários devem inserir os valores das variáveis simbólicas usadas nas funções simbólicas. O AutoGenU.ipynb gera então os arquivos nmpcmodel.cpp e nmpcmodel.hpp, que descrevem uma classe C++ NMPCModel representando as configurações do problema NMPC, incluindo $f(x, u, p)$, $C(x, u, p)$, $\dot{y}(x, p)$, $H(x, u, \dot{y}, \ddot{y}, p)$, $H_u(x, u, \dot{y}, \ddot{y}, p)$, $\dot{y}(x, u, \dot{y}, p)$ e os respectivos parâmetros. A quarta parte consiste em "construir e executar a simulação", na qual os usuários devem selecionar qual solucionador NMPC usar, inserir os parâmetros para o solucionador e os parâmetros de entrada para a simulação numérica. O arquivo Auto-GenU.ipynb gera o main.cpp, que chama o solucionador numérico baseado em C/GMRES, define os parâmetros do solucionador e chama uma função para executar as simulações numéricas. Um script para a compilação dos arquivos-fonte C++, o CMakeLists.txt, também é gerado na mesma etapa. Em seguida, os arquivos-fonte C++ são compilados através do CMakeLists.txt e as simulações numéricas são executadas. Após as simulações numéricas, são gerados arquivos .dat que armazenam o histórico temporal dos resultados da simulação. A última etapa consiste na "desenho de gráficos", na qual os gráficos dos resultados da simulação são desenhados com base nos arquivos .dat.

Solucionadores numéricos de NMPC baseados em C/GMRES Os três solucionadores de NMPC a seguir estão disponíveis no diretório src/solver:

- O método C/GMRES original (disparo único).
- O método C/GMRES baseado em disparos múltiplos com condensação de Xy .
- O método C/GMRES baseado em disparos múltiplos com condensação de Xy e U referente a (39).

Para usar esses solucionadores, a classe do solucionador precisa incluir arquivos de código-fonte C++ que definem uma classe NMPCModel, que possui as funções-membro mostradas na Figura 2. As funções-membro públicas stateFunc, phixFunc e hxFunc na Figura 2 correspondem a $f(x, u, p)$, $\dot{y}(x, p)$ e $H(x, u, \dot{y}, \ddot{y}, p)$, respectivamente. huFunc na Figura 2 denota um vetor composto por $H_u(x, u, \dot{y}, \ddot{y}, p)$, $C(x, u, p)$ e $\dot{y}(x, u, \dot{y}, p)$. Outras funções-membro públicas, dimState e dimControlInput, retornam as dimensões de x e u , e dimConstraints retorna a dimensão total de $C(x, u, p)$ e $h(x, u, p)$, respectivamente. Observe que o arquivo AutoGenU.ipynb gera automaticamente

```
classe NMPCModel { privado:
```

```
...
```

```
público:
```

```
void stateFunc(const double t, const double* x,
               const double* u, double* f);
void phixFunc(const double t, const double* x,
              double* phix);
void hxFunc(const double t, const double* x,
            const double* u, const double* lmd double* hx);

void huFunc(const double t, const double* x,
            const double* u, const double* lmd double* hu); int dimState(),
const; int
dimControlInput() const; int
dimConstraints() const;
```

```
};
```

Figura 2. Funções-membro necessárias do modelo NMPC

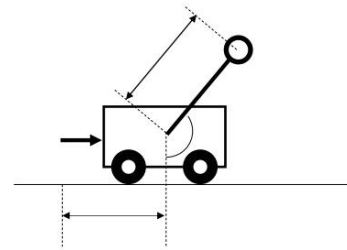


Figura 3. Haste do carrinho

nmpc model.hpp e nmpc model.cpp, que descrevem o NMPCModel.

5. EXEMPLO DE APLICAÇÃO DO AUTOGENU PARA JUPYTER

Nesta seção, apresentamos um exemplo de geração de código e simulação numérica usando o AutoGenU para Jupyter.

Consideramos o controle de elevação de uma haste de carrinho, ilustrado na Fig. 3, utilizando o método C/GMRES de disparo único. O controle de elevação da haste no carrinho é um problema complexo devido à sua subatuação e à alta não linearidade de sua dinâmica. O vetor de estado da haste do carrinho é dado por $x := \dot{y} \ y \ \ddot{y}$ e a equação é dada por $\ddot{x} = f(x, u)$.

$\dot{y} \ y \ T$, e o estado

$$\ddot{x} = \begin{pmatrix} \ddot{y} \\ \dot{y} \\ y \end{pmatrix} = \begin{pmatrix} \frac{mp \sin \dot{y} (l \dot{m} c + mp \ddot{y}^2 + g \cos \dot{y})}{(mc + mp)g \sin \dot{y} \ddot{y} \cos \dot{y} \ddot{y} \dot{m} l (mc + mp \sin^2 \dot{y})} u + \dots \end{pmatrix} \quad (40)$$

Assumimos os parâmetros físicos da haste do carrinho como $mc = 2$ [kg], $mp = 0,2$ [kg] e $l = 0,5$ [m]. Consideramos impor limites à entrada de controle, ou seja, $u_{min} \leq u \leq u_{max}$ com $u_{min} = -15$, $u_{max} = 15$. Introduzimos uma entrada fictícia $\ddot{y} \ R$ e transformamos essas restrições de desigualdade na restrição de igualdade da forma (39). O objetivo é inverter a haste e estabilizar o carrinho, ou seja, fazer o


```

dimx = 4
dimu = 2
dimc = 1
dimh = 0

t = Symbol('t')
x = symbols('x[0:%d]' % (dimx))
u = symbols('u[0:%d]' % (dimu+dimc+dimh))
lmd = symbols('lmd[0:%d]' % (dimx))

# Define user variables used in the state function here
m_c, m_p, l, g = symbols('m_c, m_p, l, g')

# variables for constraints on the control input
u_min, u_max, dummy_weight = symbols('u_min, u_max, dummy_weight')

q = symbols('q[0:%d]' % (dimx))
r = symbols('r[0:%d]' % (dimu+dimc))
q_terminal = symbols('q_terminal[0:%d]' % (dimx))
x_ref = symbols('x_ref[0:%d]' % (dimx))
fb_eps = symbols('fb_eps[0:%d]' % (dimh))

# Define the state equation
fxu = [x[2],
        x[3],
        (u[0] + m_p*sin(x[1])*(l*x[1]*x[1] + g*cos(x[1])))/(m_c+m_p*sin(x[1])
        -u[0]*cos(x[1]) - m_p*l*x[1]*x[1]*cos(x[1])*sin(x[1]) - (m_c+m_p)

# Define the constraints (if dimc > 0)
Cxu = [u[0]**2 + u[1]**2 - ((u_max-u_min)**2)/4]

# Define the inequality constraints h(x, u) <= 0 (if dimh > 0) considered
hxu = []

# Define the stage cost
L = sum(q[i]*(x[i] - x_ref[i])**2 for i in range(dimx))/2 + (r[0] * u[0] +
    - dummy_weight*u[1])

# Define the terminal cost
phi = sum(q_terminal[i]*(x[i] - x_ref[i])**2 for i in range(dimx))/2

```

Figura 4. Parte “Descrição do problema” do arquivo AutoGenU.ipynb

```

Hamiltonian = L + symfunc.dot_product(lmd, fxu) + sum(u[dimu+i] * Cxu[i]
phix = symfunc.diff_scalar_func(phi, x)
hx = symfunc.diff_scalar_func(Hamiltonian, x)
hu = symfunc.diff_scalar_func(Hamiltonian, u)
for i in range(dimh):
    hu[dimu+dimc+i] = sqrt(u[dimu+dimc+i]**2 + hxu[i]**2 + fb_eps[i]) - l

symfunc.simplify_vector_func(phix)
symfunc.simplify_vector_func(hx)
symfunc.simplify_vector_func(hu)

```

Figura 5. Parte de “computação simbólica” do AutoGenU.ipynb

O estado converge para $x_{ref} = [0 \text{ } \dot{y} \text{ } 0 \text{ } 0]^T$. Para isso, definimos o custo terminal como 1.

$$\dot{y}(x) = 2 - (x \dot{y} x_{ref}) TQ(x \dot{y} x_{ref}), \quad (41)$$

onde $Q = \text{diag} \{2.5, 10, 0.01, 0.01\}$, e o custo do estágio como 1

$$L(x, u) = 2 \text{ onde } (x \dot{y} x_{ref}) TQ(x \dot{y} x_{ref}) + \frac{1}{2} ru^2 \dot{y} \hat{r} u, \quad (42)$$

$r = 1$, $\hat{r} = 0.1$. Observe que o último termo em (42) serve para determinar o sinal de $\dot{u}$ de forma única e evitar falhas no cálculo numérico (ver Ohtsuka (2004)).

As entradas do usuário para essas definições de problema no arquivo Auto-GenU.ipynb e a geração de código dos arquivos fonte C++ das configurações do problema NMPC são ilustradas nas Figuras 4, 5 e 6, correspondentes às partes de “descrição do problema”, “computação simbólica” e “geração de código”, respectivamente. Na parte de “descrição do problema”, primeiro inserimos as dimensões do estado, da entrada de controle, das restrições de igualdade e das restrições de desigualdade tratadas pela função FB semi-suave, que são definidas como $\dim_x$, $\dim_u$, $\dim_c$ e $\dim_h$, respectivamente, na primeira célula da Figura 4.

Observe que a entrada de controle agora é dada por $[u \text{ } \dot{u}]^T$ e R^2

```

scalar_parameters = [[m_c, 2], [m_p, 0.2], [l, 0.5], [g, 9.80665],
                    [u_min, -15], [u_max, 15], [dummy_weight, 0.1]]

array_parameters = [['q', dimx, '{2.5, 10, 0.01, 0.01}'],
                   ['r', dimu, '{1, 0.01}'],
                   ['q_terminal', dimx, '{2.5, 10, 0.01, 0.01}'],
                   ['x_ref', dimx, '{0, M_PI, 0, 0}']]

model_name = "cartpole"
cse_flag = True

gencpp.make_model_dir(model_name)
gencpp.generate_cpp(fxu, phix, hx, hu, model_name, cse_flag)
gencpp.generate_hpp(dimx, dimu, dimc+dimh, scalar_parameters, array_parameters)

```

Figura 6. Parte de “geração de código” do AutoGenU.ipynb

Ou seja, $\dim_u = 2$, $\dim_c$ é dado por 1 e $\dim_h$ por 0. As variáveis simbólicas t , x , u e $\dot{y}$ são então definidas em AutoGenU.ipynb, como mostrado na segunda célula da Fig. 4. Em seguida, definimos as variáveis simbólicas usadas em $f(x, u, p)$, $C(x, u, p)$, $h(x, u, p)$, $L(x, u, p)$, $\dot{y}(x, p)$, como mostrado na terceira e quarta células da Fig. 4, e definimos essas funções simbólicas, como mostrado na quinta célula da Fig. 4. Na próxima parte, ou seja, a parte de “computação simbólica”, $\dot{y}x(x, p)$, $Hx(x, u, \dot{y}, p)$, $Hu(x, u, \dot{y}, p)$ e $\dot{y}(x, u, \dot{y}, p)$ são computadas simbolicamente, como mostrado na primeira célula da Fig. 5. A segunda célula simplifica essas funções simbólicas. Na parte de “geração de código”, inserimos os valores das variáveis simbólicas, incluindo os parâmetros físicos da haste do carrinho e os parâmetros de peso na função de custo, conforme mostrado na primeira célula da Figura 6. Também definimos o nome do diretório onde os arquivos-fonte C++ são gerados e decidimos se usaremos a eliminação de subexpressões comuns (CSE) na geração de código por meio da flag ‘cse’. Se a flag ‘cse’ for ‘True’, novas variáveis são introduzidas para substituir os termos comuns nos códigos gerados, reduzindo assim o tempo de computação. Após essas configurações, ‘gencpp.generate cpp’ e ‘gencpp.generate hpp’ geram os arquivos ‘nmpc model.cpp’ e ‘nmpc model.hpp’, que contêm a classe ‘NMPCModel’. A Figura 7 mostra um exemplo do arquivo ‘nmpc model.cpp’ gerado com a flag ‘cse’ definida como ‘True’. Na Figura 7, ‘x0’, ‘...’, ‘x4’ são introduzidos para a CSE. O modelo nmpc gerado (nmpc.model.hpp) define o NMPCModel, conforme mostrado na Figura 2. Observe que o modelo nmpc gerado (nmpc.model.hpp) também contém os parâmetros físicos do poste do carrinho e os parâmetros de peso na função de custo como variáveis de membro privadas.

A seguir, apresentamos a parte de “construção e execução da simulação”, ilustrada na Figura 8. Primeiramente, escolhemos qual solucionador usar para implementar o NMPC: o método C/GMRES original, o método C/GMRES baseado em múltiplos disparos com condensação de $X\dot{y}$ ou aquele com condensação de $X\dot{y}$ e U . Neste exemplo, escolhemos o método C/GMRES original, como mostrado na primeira célula da Figura 8. Em seguida, precisamos definir os parâmetros para o solucionador, os parâmetros para a inicialização da solução do NMPC e os parâmetros para a simulação numérica. Definimos os parâmetros para o horizonte em (30) como $T_f = 2$ e $\dot{y} = 1$, o número de discretizações do horizonte como $N = 100$, o parâmetro de estabilização como $\dot{y} = 1000$, o incremento de diferença finita para a aproximação de diferença como $h = 1.0 \times 10^{-8}$ e o número máximo de iterações do GMRES como $k_{max} = 10$. Essas configurações de parâmetros são mostradas na segunda célula da Fig. 8. Os parâmetros para a inicialização da solução são necessários para que o método de Newton resolva (31). Definimos o palpite inicial da solução do método de Newton como $[0.1, 0.1, 0.1]^T$, resid-

```
#include "nmnpc_model.hpp"

void NMPCModel::stateFunc(const double t, const double*
    x, const double* u, double* f) {
    double x0 = sin(x[1]);
    double x1 = 1.0/(m_c + m_p*pow(x0, 2));
    double x2 = cos(x[1]);
    double x3 = l*pow(x[1], 2);
    double x4 = m_p*x0;
    f[0] = x[2];
    f[1] = x[3];
    f[2] = x1*(u[0] + x4*(g*x2 + x3));
    f[3] = x1*(-g*x0*(m_c + m_p) - vcc[0]*x2 - x2*x3*x4)/l
    ;
}

void NMPCModel::phixFunc(const double t, const double* x
    , double* phix) {
    ...
}

void NMPCModel::hxFunc(const double t, const double* x,
    const double* u, const double* lmd , duplo* hx) {
    ...
}

void NMPCModel::huFunc(const double t, const double* x,
    const double* u, const double* lmd , duplo* hu) {
    ...
}
```

Figura 7. Exemplo de modelo nmnpc.cpp gerado com configuração Sinalizador cse definido como Verdadeiro. Detalhes omitidos.

```
solver_index = 1

T_f = 2.0
alpha = 1.0
N = 100
finite_difference_increment = 1.0e-08
zeta = 1000 # zeta must be the reciprocal of the sampling period
kmax = 10

solver_parameters = slvprm.SolverParameters(T_f, alpha, N, finite_differenc

initial_guess_solution = [0.01, 10, 0.01]
newton_residual_tolerance = 1.0e-06
max_newton_iteration = 50

initialization_parameters = iniprm.InitializationParameters(initial_gues
    newton_residu
    max_newton_it

initial_time = 0
initial_state = [0, 0, 0, 0]
simulation_time = 10
sampling_time = 0.001

simulation_parameters = simprm.SimulationParameters(initial_time,
    initial_state,
    simulation_time,
    sampling_time)

gencpp.generate_main(solver_index, solver_parameters, initialization_p
    simulation_parameters, model_name)
gencpp.generate_cmake(solver_index, model_name)
gencpp.generate_cmake_for_model(model_name)

cppexe.set_cmake(model_name)
cppexe.make_and_run(model_name)
```

Figura 8. Parte “Construir e executar simulação” do Auto-GenU.ipynb

ual de tolerância do método de Newton como $1,0 \times 10^{-6}$, e o número máximo de iterações de Newton é 50, conforme ilustrado na terceira célula da Fig. 8. Observe que a solução do método de Newton é composta por u , $\dot{u}$ e o multiplicador de Lagrange em relação a (39), ou seja, sua di-

```
plot = simplt.SimulationPlotter(model_name)
plot.set_scales(2,5,2)
plot.save_plots()
```

Figura 9. Parte “Desenho de gráfico” do AutoGenU.ipynb

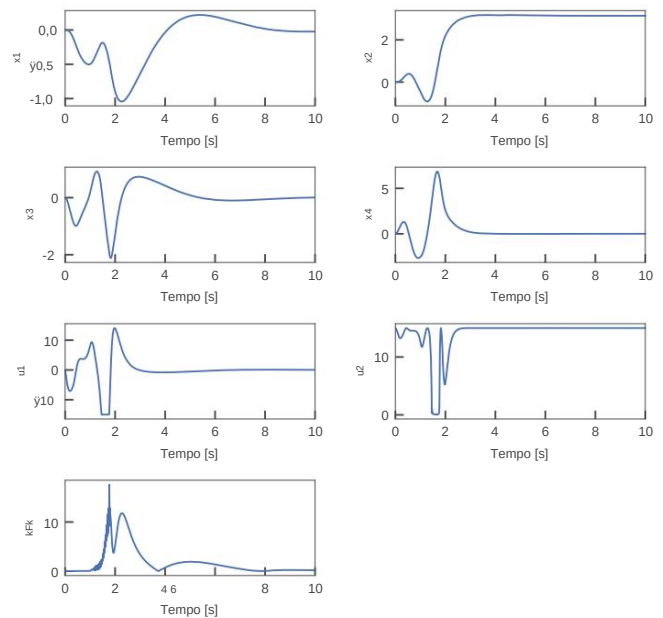


Figura 10. Resultados da simulação do controle da haste do carrinho.

A menção é dada por 3. Neste exemplo, realizamos 10-s simulação numérica definindo o período de amostragem como 1 ms e estado inicial como $[0 \ 0 \ 0 \ 0]^T$. Os parâmetros Para simulação numérica, os dados são inseridos no arquivo AutoGenU.ipynb, como mostrado na quarta célula da Fig. 8. Após definir esses parâmetros, o arquivo main.cpp chama uma função para executar o simulações numéricas e o arquivo CMakeLists.txt são gerados. por gencpp.generate_main, gencpp.generate_cmake e gencpp.generate_cmake para o modelo na quinta célula da Fig. 8. A última célula da Figura 8 então constrói códigos C++ com base em O arquivo CMakeLists.txt é executado e simulações numéricas são realizadas.

A Figura 9 mostra a parte do “desenho do gráfico”. Nesta parte, Definimos os tamanhos do gráfico, da fonte e os espaços entre os elementos. gráficos usando plot.set scales. A Figura 10 mostra o Resultados da simulação do controle de elevação do poste do carrinho Desenhado por AutoGenU.ipynb. Nesta figura, u_1 é o real O sinal de controle é aplicado ao eixo do carrinho, u_2 é o indicador. entrada, e “F” é a norma de (25). Podemos com sucesso Levante o poste satisfazendo as restrições de desigualdade. na entrada de controle. O tempo médio de computação de A atualização de controle é de 0,18 ms em um Intel Core i5 de 1,6 GHz. CPU, e conseguimos NMPC em tempo real.

Neste artigo, apresentamos apenas um exemplo de controle. o poste do carrinho usando o método C/GMRES de disparo único. Katayama (2018-2020) fornece outros exemplos usando o métodos baseados em disparos múltiplos e o FB semi-suave função.

6. CONCLUSÕES

Apresentamos uma ferramenta de NMPC com interface amigável. interface utilizando JupyterLab e Jupyter Notebook, AutoGenU para Jupyter. Esta ferramenta fornece geração automática de gerações de software.

Geração de código-fonte C++ para arquivos que representam configurações de problemas NMPC usando o pacote de computação simbólica SymPy. Também suporta a compilação de arquivos-fonte C++ com CMakeLists.txt, executa simulações numéricas e cria gráficos dos resultados da simulação. Trabalhos futuros incluem a aplicação dos solucionadores NMPC baseados em C/GMRES ao ROS/ROS2 para facilitar a implantação desses solucionadores em sistemas reais. A paralelização do método C/GMRES baseado em disparos múltiplos também está prevista para trabalhos futuros, visando aumentar sua velocidade.

REFERÊNCIAS

- Andersson, JAE, Gillis, J., Horn, G., Rawlings, JB e Diehl, M. (2019). CasADi – Uma estrutura de software para otimização não linear e controle ótimo. *Mathematical Programming Computation*, 11(1), 1–36.
- Bock, HG e Plitt, KJ (1984). Um algoritmo de disparo múltiplo para solução direta de problemas de controle ótimo. *Volumes de Atas da IFAC*, 17(2), 1603 – 1608.
- Bryson, AE e Ho, YC (1975). *Controle Ótimo Aplicado: Otimização, Estimação e Controle*. CRC Press.
- Deng, H. e Ohtsuka, T. (2018). Um conjunto de ferramentas de geração de código paralelo para controle preditivo de modelo não linear. Em 2018 IEEE Conference on Decision and Control (CDC), 4920–4926.
- Deng, H. e Ohtsuka, T. (2019). Um método paralelo do tipo Newton para controle preditivo de modelo não linear. *Automatica*, 109, 108560.
- Diehl, M., Bock, H., e Schlöder, JP (2005). Um esquema de iteração em tempo real para otimização não linear em controle de feedback ótimo. *SIAM J. Control and Optimization*, 43, 1714–1736.
- Gifftalher M., et al. (2018). The Control Toolbox - uma biblioteca C++ de código aberto para robótica, controle ótimo e preditivo baseado em modelos. Em 2018 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN), 123–129.
- Granger, B. e Grout, J. (2016). JupyterLab: Blocos de construção para computação interativa. URL <https://jupyterlab.readthedocs.io/en/latest/>.
- Herceg, M., Kvasnica, M., Jones, CN e Morari, M. (2013). Multi-Parametric Toolbox 3.0. In *European Control Conference*, 502–510.
- Houska, B., Ferreau, HJ e Diehl, M. (2011). ACADO Toolkit – Uma estrutura de código aberto para controle automático e otimização dinâmica. *Optimal Control Applications and Methods*, 32(3), 298–312.
- Hunter, JD (2007). Matplotlib: Um ambiente gráfico 2D. *Computação em Ciência e Engenharia*, 9(3), 90– 95.
- Katayama, S. (2018-2020). AutoGenU para Jupyter. URL <https://github.com/mayataka/autogenu-jupyter>.
- Kelly, CT (1995). Métodos iterativos para equações lineares e não lineares. *Fronteiras em matemática aplicada*. SIAM.
- Kluyver T., et al. (2016). Jupyter Notebooks – um formato de publicação para fluxos de trabalho computacionais reproduzíveis. Em *Posicionamento e Poder na Publicação Acadêmica: Jogadores, Agentes e Agendas*, 87 – 90.
- Liao-McPherson, D., Huang, M., e Kolmanovsky, I. (2019). Um método de Fischer-Burmeister regularizado e suavizado para programação quadrática com aplicações.
- para modelar o controle preditivo. *IEEE Transactions on Automatic Control*, 64(7), 2937–2944.
- Magni, L., Raimondo, DM, e Allgöwer, F. (2008). *Controle preditivo de modelo não linear: rumo a novas aplicações desafiadoras*, volume 384. Springer.
- Meurer A., et al. (2017). SymPy: computação simbólica em Python. *PeerJ Computer Science*, 3, e103.
- Nocedal, J. e Wright, SJ (2006). *Otimização Numérica*. Springer, segunda edição.
- Ohtsuka, T. (2004). Um método de continuação/GMRES para computação rápida de controle de horizonte deslizante não linear. *Automática*, 40(4), 563 – 574.
- Ohtsuka, T. (2015). Um tutorial sobre C/GMRES e geração automática de código para controle preditivo de modelo não linear. Em 2015 European Control Conference (ECC), 73–86.
- Ohtsuka, T. e Fujii, HA (1997). Algoritmo de otimização em tempo real para controle de horizonte deslizante não linear. *Automática*, 33(6), 1147 – 1154.
- Ohtsuka, T. e Kodama, A. (2002). Sistema de geração automática de código para controle de horizonte deslizante não linear. *Transações da Sociedade de Engenheiros de Instrumentação e Controle*, 38(7), 617–623.
- Pérez, F. e Granger, BE (2007). IPython: um sistema para computação científica interativa. *Computação em Ciência e Engenharia*, 9(3), 21–29.
- Richter, SL e DeCarlo, RA (1983). Métodos de continuação: Teoria e aplicações. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(4), 459– 464.
- Risbeck, MJ e Rawlings, JB (2015). MPCTools: Ferramentas de controle preditivo de modelo não linear para CasADi (interface Python). URL <https://bitbucket.org/rawlings-group/mpc-tools-casadi>.
- Shimizu, Y., Ohtsuka, T., e Diehl, M. (2009). Um algoritmo em tempo real para controle de horizonte recuado não linear usando disparo múltiplo e método de continuação/Krylov. *Revista Internacional de Controle Robusto e Não Linear*, 19(8), 919–936.
- Sideris, A. e Bobrow, JE (2005). Um algoritmo linear quadrático sequencial eficiente para resolver problemas de controle ótimo não linear. Em *Anais da Conferência Americana de Controle de 2005*, 2275–2280 vol. 4.
- Tassa, Y., Erez, T., e Smart, B. (2008). Programação dinâmica diferencial de horizonte recuado. Em *Advances in Neural Information Processing Systems* 20, 1465– 1472.
- Tassa, Y., Erez, T., e Todorov, E. (2012). Síntese e estabilização de comportamentos complexos através da otimização de trajetória online. Em 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 4906–4913.
- Waskom M., et al. (2012-2019). seaborn: visualização de dados estatísticos. URL <https://seaborn.pydata.org/>.